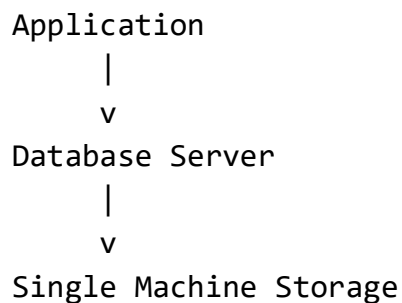


Big Data Systems and Hadoop Fundamentals

1 Traditional System Architecture

Example architecture:



This type of architecture is called a **Monolithic System**.

Definition :

A **monolithic system** is a system where **one machine handles everything**.

That machine is responsible for:

- Storage
- Processing
- Queries
- Data management

Example:

An online retail company storing data in MySQL.

Orders Table

Users Table

Products Table

Everything runs on **one server**.

Problems With Monolithic Systems

3 major problems.

1 Storage Limit

A single machine may have:

4 TB

8 TB

10 TB

But companies generate:

100 TB

1 PB

10 PB

So one machine cannot store all data.

2 Processing Limit

Example query:

```
SELECT COUNT(*) FROM transactions;
```

If transactions table has:

2 billion rows

One server becomes very slow.

3 Hardware Failure

If the server fails:

System stops
Data becomes unavailable

This creates downtime.

Introduction to Distributed Systems

To solve this problem, companies started using **multiple machines connected together**.

Example:

Machine1
Machine2
Machine3
Machine4

Each machine contributes:

Storage
CPU
Memory

This collection of machines is called a **Cluster**.

Definition :

A **cluster** is a group of machines connected together that work as a single system.

This architecture is called a **Distributed System**.

Hadoop Overview

Hadoop is a framework that allows us to:

- Store large data across machines
- Process data in parallel

- Handle failures automatically

Hadoop has three main components.

HDFS

MapReduce

YARN.

Component 1 – HDFS

HDFS stands for:

Hadoop Distributed File System

Purpose:

Storage

Large files are stored across multiple machines.

Component 2 – MapReduce

Purpose:

Processing / Computation

Used to process large datasets in parallel.

Example:

Word Count

Log Analysis

Sales Aggregation

Component 3 – YARN

Full form:

Yet Another Resource Negotiator

Purpose:

Resource management

It decides:

Which machine runs which job

CPU allocation

Memory allocation

HDFS Architecture

HDFS Architecture Model

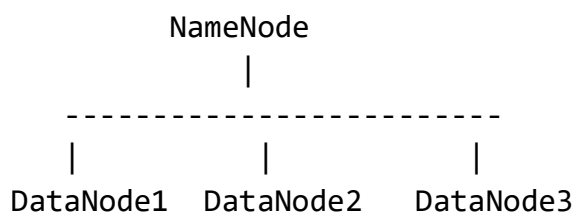
HDFS follows a **Master – Slave architecture**.

Explain this clearly.

Master Node → NameNode

Slave Nodes → DataNodes

Diagram you can draw on screen:



Role of NameNode

The NameNode stores **metadata**.

Metadata means:

information about the data

Example metadata:

File name

Block locations

Replication factor

Permissions

Example metadata table:

File	Block	Location
file1.txt	B1	DN1
file1.txt	B2	DN2
file1.txt	B3	DN3

Important point:

The NameNode **does NOT** store actual data.

Role of DataNodes

DataNodes store the **actual file blocks**.

Example:

file1.txt

Stored as:

Block1 → DataNode1

Block2 → DataNode2

Block3 → DataNode3

HDFS Block Concept

Hadoop divides files into **blocks**.

Block size depends on Hadoop version.

Hadoop 1 → 64 MB

Hadoop 2 / 3 → 128 MB

Example file:

300 MB file

Blocks:

Block	Size
B1	128MB
B2	128MB
B3	44MB

Important concept to explain:

If the last block only contains 44MB, the remaining space in that block (128MB - 44MB) is allocated to new block

A new block will start for the next file.

Replication Factor

Replication factor defines **how many copies of each block are stored**.

Default value:

3

Example:

Block1 stored in DN1 DN2 DN3

Example layout:

File1.txt

DN1	DN2	DN3	DN4
B1	B2	B3	
	B1	B2	
B3		B1	

This means every block exists **three times**.

Why Replication Is Needed

What happens if a machine goes down?

Example:

DataNode2 fails

Block copy still exists on:

DN1

DN3

So the data is still available.

Storage Tradeoff

Replication increases storage usage.

Example:

1 TB data

Replication factor = 3

Storage used = 3 TB

So we compromise between:

Storage cost vs Data reliability

Rack Awareness

data center level protection.

Machines are organized into **racks**.

Example:

Rack1 → DN1 DN2 DN3

Rack2 → DN4 DN5 DN6

Hadoop stores blocks like this:

2 copies in local rack

1 copy in remote rack

Reason:

Rack failure protection

Network efficiency

What Happens When DataNode Fails

If a DataNode fails:

Hadoop detects missing blocks

Replication factor automatically restored.

Example:

Block B1 had 3 copies

One node failed

Now only 2 copies exist

Hadoop creates a new replica on another node.

Replication factor is **always maintained**.

What Happens When DataNode Comes Back

If a dead node comes back:

Hadoop treats it as a fresh node

It is used for **future storage operations**.

NameNode Failure (Hadoop 1)

NameNode is a **Single Point of Failure**.

Term:

SPOF → Single Point Of Failure

Hadoop 1 solution:

Secondary NameNode

Secondary NameNode stores:

fsimage
edit logs

Explanation:

fsimage → snapshot of filesystem metadata
edit logs → changes to metadata

Admin manually converts SNN to NN if failure occurs.

This creates **temporary downtime**.

Hadoop 2 High Availability

To avoid downtime, Hadoop 2 introduced:

Standby NameNode

Active NameNode and Standby NameNode run together.

Coordination handled by:

ZooKeeper

ZooKeeper stores configuration and coordinates cluster nodes.

ZooKeeper Leader Election

ZooKeeper follows:

Leader – Follower architecture

Important concept:

ZooKeeper nodes must be **odd numbers**.

Example:

3 nodes

5 nodes

7 nodes

Reason:

Leader election must avoid tie

One node becomes leader.

Other nodes follow the leader.

Memory Management and Small File Problem

Block Memory Usage

Example file:

300 MB

Blocks:

128 MB

128 MB

44 MB

Don't forget to tune the block size.

Block size is configurable.

Admin can change block size in configuration files.

Block size must be **power of 2**.

Examples:

64MB
128MB
256MB
512MB

Impact of Block Size

Large block size:

less metadata
less load on NameNode

Small block size:

more metadata
higher load on NameNode

Small File Problem

Example scenario:

1 million files
each file = 2MB

Each file requires metadata entry.

Metadata stored in NameNode memory.

Approximate metadata overhead:

~150 bytes per file

Millions of small files create heavy NameNode load.

Solution – Compaction

Compaction means combining small files.

Example:

Instead of:

`1M files`

Create:

`10 large files`

This reduces metadata entries.

However sometimes compaction is not recommended.

Example:

`Files require individual tracking`

Admin decides when compaction should occur.

HDFS CLI Commands

`docker-compose up -d`

`docker-compose down -v`

Open Hadoop container.

`docker exec -it namenode bash`

`hdfs dfsadmin -report`

Section	Meaning
Configured Capacity	Total storage across nodes
DFS Used	Storage currently used
Live datanodes	Number of storage machines

Example output:

Live datanodes (2)

STEP 1: Check Hadoop Version

`hadoop version`

Step 2 – Check HDFS Root Directory

Run:

`hdfs dfs -ls /`

Explain:

This command lists directories in the Hadoop filesystem.

Similar to Linux command:

`ls /`

At first it may show:

Found 0 items

Because HDFS is empty.

Step 3 – Create Directory in HDFS

Create a folder called **data**.

```
hdfs dfs -mkdir /data
```

Verify:

```
hdfs dfs -ls /  
/data
```

Explain:

HDFS directory structure behaves like Linux filesystem.

Step 4 – Create Sample Dataset

Now create a small dataset locally.

```
echo "order_id,product,amount" > sales.csv  
echo "1,Phone,800" >> sales.csv  
echo "2,Laptop,1200" >> sales.csv  
echo "3,Mouse,25" >> sales.csv  
echo "4,Keyboard,100" >> sales.csv  
echo "5,Monitor,400" >> sales.csv
```

Show the file:

```
cat sales.csv  
order_id,product,amount  
1,Phone,800  
2,Laptop,1200  
3,Mouse,25  
4,Keyboard,100  
5,Monitor,400
```

Explain:

This file currently exists in the container's local filesystem.

Step 5 – Upload File to HDFS

Upload file to Hadoop.

```
hdfs dfs -put sales.csv /data
```

Explain:

This command copies file from local system → HDFS.

Verify upload.

```
hdfs dfs -ls /data
```

Expected output:

```
sales.csv
```

Step 6 – Read File from HDFS

Run:

```
hdfs dfs -cat /data/sales.csv
```

Explain:

This reads file stored in distributed filesystem.

Step 7 – Copy File Inside HDFS

Create a backup folder.

```
hdfs dfs -mkdir /backup
```

Copy file.

```
hdfs dfs -cp /data/sales.csv /backup
```

Verify:

```
hdfs dfs -ls /backup
```

Command	Meaning
---------	---------

cp	copy
----	------

mv	move
----	------

Step 8 – Move File

Move file from backup to archive.

First create archive folder.

```
hdfs dfs -mkdir /archive
```

Move file.

```
hdfs dfs -mv /backup/sales.csv /archive
```

Verify:

```
hdfs dfs -ls /archive
```

Step 9 – Change Replication Factor

Check current replication.

```
hdfs fsck /data/sales.csv -files -blocks
```

Now change replication factor.

```
hdfs dfs -setrep 2 /data/sales.csv
```

Verify again.

```
hdfs fsck /data/sales.csv -files -blocks
```

Explain:

Replication factor controls number of block copies.

Step 10 – Check Disk Usage

Run:

```
hdfs dfs -du /data
```

Example output:

```
34 102 /data/sales.csv
```

Explain:

Column	Meaning
File size	actual file size
Disk usage	size including replication

Step 11 – Create Empty File

Run:

```
hdfs dfs -touchz /data/empty.txt
```

Verify:

```
hdfs dfs -ls /data
```

Explain:

Creates a zero-byte file.

Step 12 – Append Files

Create another file.

```
echo "6,Tablet,500" > extra.csv
```

Append to existing file.

```
hdfs dfs -appendToFile extra.csv /data/sales.csv
```

View result.

```
hdfs dfs -cat /data/sales.csv
```

Step 13 – Merge Files from HDFS

Create multiple files.

```
hdfs dfs -put sales.csv /data/file1
```

```
hdfs dfs -put sales.csv /data/file2
```

Merge files into local file.

```
hdfs dfs -getmerge /data merged.csv
```

Check merged file.

```
cat merged.csv
```

Explain:

getmerge combines multiple HDFS files into one Linux file.

Step 14 – CRC File Explanation

When file is downloaded:

merged.csv
merged.csv.crc

Explain:

CRC stands for **Cyclic Redundancy Check**.

It verifies **data integrity in distributed systems**